

Exercise 9: Catching Confident Wrongness with Tests and Hooks

DEV CYCLE	Assure (testing + guardrails)
CC FEATURES	pytest generation from a spec, mocking the Claude API, spotting hallucinated APIs, mutation checks, Claude Code hooks (PreToolUse, PostToolUse), /hooks
SPEC TRACES	SPEC §12 traceability matrix, NFR-03 (offline tests), SEC-06 (hooks)
TIME	35 min
YOU PRODUCE	A passing, offline pytest suite with one test per requirement, plus project hooks that protect secrets and run the suite after every edit

Objective

AI is confidently wrong in specific ways: invented library functions, fabricated endpoints, logic that looks right and fails at the edge. The defence is tests, and in a spec-driven project the tests are generated from the spec, not from the code, so they check what was agreed rather than what was written. Then you will make green tests a precondition for trust, mechanically: Claude Code hooks that run the suite after every file edit and block edits to secrets and the database.

Walk-through

1 Generate the suite from the traceability matrix

Point Claude at SPEC §12 and ask for one test per row. The Anthropic client must be faked, so the suite is fast, free and offline.

TYPE THIS PROMPT INTO CLAUDE CODE

Create tests/conftest.py and tests/test_api.py from the traceability matrix in @docs/SPEC.md §12, using the exact test names listed there. Use FastAPI TestClient against app.main:app with a fresh temporary SQLite database per test (override get_db, enable foreign keys). Provide a fixture that monkeypatches the Anthropic client used by app/drafting.py with a fake whose messages.create returns a configurable JSON string and usage, and can be told to raise an API error or return invalid JSON. Load sample records from samples/records/. Cover rows FR-01 to FR-14, FR-20, AI-02, AI-04, AI-05, SEC-01, SEC-02 and SEC-03 (skip Exercise 10 rows for now). Use only real pytest and FastAPI APIs. Each test docstring states the requirement it proves.

VALIDATE Read the generated files. Every test name matches SPEC §12; every assertion maps to an acceptance criterion; the fake client means no test touches the network.

test_sec03_payload_has_no_phi should inspect the arguments captured by the fake client, not the draft output.

2 Run the suite

The first run rarely passes cleanly, and that is the point: tests expose the gap between what Claude claimed and what the code does.

RUN IN TERMINAL

```
PS> python -m pytest -q
```

VALIDATE You get a pass/fail summary. Classify every failure: a real bug in the app, a test that asserts something the spec does not say, or a test that uses an API that does not exist.

3 Hunt for hallucinated APIs

If a test fails on an `AttributeError` or "unexpected keyword argument", Claude may have invented a helper. Confront it directly, with the spec as the referee.

TYPE THIS PROMPT INTO CLAUDE CODE

This test fails with [paste error]. Is this (a) a real bug in my code, (b) a test asserting behaviour the spec does not require, or (c) a pytest, FastAPI or anthropic API that does not actually exist? Verify against the real library, quote the SPEC requirement, and fix only the thing that is wrong. Do not weaken an assertion just to make it pass.

VALIDATE Claude distinguishes a genuine bug from a fabricated API or an over-reaching test, and fixes the right one. "Do not weaken an assertion" matters: making a failing test pass by deleting its assertion is the most common confident-wrongness move of all.

4 Prove the tests can fail

Tests that never fail are decoration. Break the most important safety rule on purpose and make sure the suite notices, then use `rewind` to undo the sabotage.

TYPE THIS PROMPT INTO CLAUDE CODE

Temporarily comment out the four-eyes check in the approve route, run `python -m pytest -q`, and show me which tests fail. Then restore the check exactly as it was.

VALIDATE `test_fr10_four_eyes` fails while the check is gone and passes once it is back. If nothing fails, the test is asserting nothing useful: ask Claude to strengthen it. If the restore is not exact, use `/rewind` to return to the checkpoint before the sabotage.

5 Add the Claude Code hooks

Hooks are shell commands Claude Code runs automatically at points in its loop. A `PreToolUse` hook runs before a tool and can block it by exiting with code 2; a `PostToolUse` hook runs after, and exit code 2 feeds its message back to Claude. You will add one guard and one quality gate.

TYPE THIS PROMPT INTO CLAUDE CODE

Create `.claude/hooks/protect_files.py`: read the hook JSON from stdin, get `tool_input.file_path`, and if the path is `.env`, ends with `.db`, or is inside `.git/`, print a clear reason to stderr and exit 2 (block); otherwise exit 0. Create `.claude/hooks/run_tests.py`: run `python -m pytest -q -x`, and if it fails print the last 30 lines to stderr and exit 2 so you see the failure; otherwise exit 0. Then create `.claude/settings.json` registering them exactly like this: [paste the JSON below]. Standard library only.

CREATE / PASTE → .CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Edit|MultiEdit|Write",
        "hooks": [{ "type": "command", "command": "python .claude/hooks/protect_files.py"
      } ]
    },
    "PostToolUse": [
      { "matcher": "Edit|MultiEdit|Write",
        "hooks": [{ "type": "command", "command": "python .claude/hooks/run_tests.py" } ]
      }
    ]
  }
}
```

VALIDATE Exit and restart `claude` so the new settings load, then run `/hooks`: both hooks are listed. Commit `.claude/settings.json` and the hook scripts: they are part of the project, and every teammate gets the same guardrails.

INSIDE THE CLAUDE CODE SESSION

```
> /hooks
```

6 Watch the hooks work

Test both directions: the guard must block, and the gate must catch a real regression.

PROMPT 1 · THE GUARD SHOULD BLOCK THIS

Add `DEBUG=true` to the `.env` file.

PROMPT 2 · THE TEST GATE SHOULD CATCH THIS

In `app/schemas.py` change the minimum reject reason length from 10 to 2.

VALIDATE The first request is blocked by `protect_files.py` with your reason, and Claude explains it cannot edit `.env`. The second edit goes through, the `PostToolUse` hook runs `pytest`, `test_fr09_reject_requires_reason` fails, and Claude sees the failure and offers to revert. Let it revert and confirm the suite is green again. Green is now enforced, not hoped for.

SCREENSHOT SLOT *PostToolUse hook output: pytest failure fed back to Claude after the schema edit*

What success looks like

- An offline `pytest` suite with test names that match SPEC §12, and it passes twice in a row.
- You found and corrected at least one place where the first version did not do what it claimed.
- You proved a key test can fail by sabotaging the code it protects.
- Project hooks block edits to `.env`, `*.db` and `.git/`, and run the suite after every edit.

Common pitfalls (Windows and macOS)

- Run `pytest` from the repo root with the `venv` active, or imports of `app` fail. Hooks inherit the environment `claude` was started in, so start it from an activated `venv`.
- A test that calls the real Anthropic API is slow, costly and flaky by design. Insist on the fake client.
- Hooks are guardrails, not a sandbox: the file guard only sees Edit and Write. A Bash command could still write to `.env`. Say so in the production-gap audit.
- Running the full suite after every edit gets slow as it grows. `-x` stops at the first failure; later you might run only affected tests.

Stretch goal

Install `pytest-cov` (ask first: it is a new dependency, and `CLAUDE.md` says so), run with `--cov=app`, and ask Claude which untested branch is riskiest for patient safety. Write one test for it. Coverage is a map of your blind spots, not a score to maximise.